Compiler Design — Complete Question Bank Answers

Exam-ready answers based primarily on the uploaded Compiler Design notes and question bank. Where the question bank says that a diagram, grammar, or regular expression will be supplied separately, the PDF gives the correct solving method rather than inventing missing data.

UNIT 1 — Introduction to Compiler

Q1. What is a Compiler? List major functions done by Compiler.

A compiler is a language processor that converts a program written in a high-level programming language (source code) into a lower-level language or machine/object code (target code). It also detects and reports errors during translation.

Major functions: (1) source-to-target translation, (2) preservation of source-program meaning, (3) error detection and reporting, (4) efficient target-code generation, and (5) object-code generation.

Q2. Compare Single-Pass and Multi-Pass Compiler.

Single-Pass Compiler	Multi-Pass Compiler
Processes the source in one major pass.	Processes the source through multiple passes.
Generally faster compilation.	Generally takes more compilation time.
Less intermediate processing.	Uses intermediate representations between passes.
Simpler compiler structure.	More complex compiler structure.
Limited opportunity for extensive optimization.	Better opportunity for analysis and optimization.

Q3. Differentiate between Compiler and Interpreter.

Compiler	Interpreter
Translates the entire source program.	Translates and executes the program statement by statement.
Produces object/machine code.	Does not generate object code in the same manner.
Initial compilation takes more time.	Execution can begin quickly.
Resulting program generally executes faster.	Execution is comparatively slower.
Errors are generally reported after analysis/compilation.	Stops when it encounters an error during execution.

Examples of interpreters: Perl, Python and MATLAB.

Q4. Differentiate between Compiler and Assembler.

Compiler	Assembler
Converts high-level language to machine/object code.	Converts assembly language to machine/object code.
Input is a high-level program.	Input is an assembly-language program.
Performs lexical, syntax and semantic analysis.	Primarily translates assembly instructions and symbols.

Q5 / 8. Explain the phases of a Compiler with a suitable diagram.

```
Source Program
      |
      v
Lexical Analysis
      |
```

```
      v
Syntax Analysis
      |
      v
Semantic Analysis
      |
      v
Intermediate Code Generation
      |
      v
Code Optimization
      |
      v
Target Code Generation
      |
      v
Target Code

(Symbol Table and Error Handler support the phases.)
```

Lexical Analysis: Reads source characters and groups them into tokens.

Syntax Analysis: Checks whether the token sequence follows the grammar and produces a parse/syntax representation.

Semantic Analysis: Checks meaning, including type compatibility and declaration-related rules.

Intermediate Code Generation: Produces an intermediate representation of the source program.

Code Optimization: Improves the intermediate code to reduce execution time, memory use, or unnecessary instructions.

Target Code Generation: Converts the optimized representation into target machine/assembly code.

Q6. Explain the functionalities provided by Preprocessor, Linker and Loader.

Preprocessor: Processes source code before compilation. Common directives include #define, #undef, #include, #ifdef, #ifndef and #if.

Linker: Combines object modules and libraries, resolves external references and produces the executable program.

Loader: Loads the executable into main memory, performs required relocation/preparation and transfers control to the program.

```
Source
  -> Preprocessor
  -> Compiler
  -> Object Code
  -> Linker
  -> Executable
  -> Loader
  -> Main Memory
```

Q7. What do you mean by Cousins of a Compiler? Explain each in brief.

Cousins of a compiler are related language-processing programs that assist in preparing, translating, linking, loading or executing programs.

Preprocessor: Processes source directives before compilation. Assembler: Converts assembly language to machine/object code. Linker: Combines object files/libraries and resolves external references. Loader: Loads executable code into memory. Interpreter: Translates and executes source statements one at a time.

UNIT 2 — Lexical Analysis

Q9. Differentiate between DFA and NFA.

DFA	NFA
For every state and input symbol, exactly one transition.	A state may have zero, one or multiple transitions for an input.
No epsilon transitions.	Epsilon transitions may be allowed.
One computation path for an input.	May have several possible computation paths.
Usually straightforward to implement directly.	Often converted to DFA for implementation.
Recognizes regular languages.	Recognizes regular languages.

Key point: DFA and NFA have the same expressive power: both recognize exactly the regular languages.

Q10. Explain Tokens, Lexemes and Patterns with examples.

Token: A category/class of lexical units, such as IDENTIFIER, NUMBER, KEYWORD or OPERATOR.

Lexeme: The actual sequence of characters in the source program that matches a token.

Pattern: The rule describing the set of lexemes belonging to a token.

```
For: count = value + 25;

Lexemes: count   =   value   +   25   ;

Token examples:
count -> IDENTIFIER
=     -> ASSIGNMENT
value -> IDENTIFIER
+     -> OPERATOR
25    -> NUMBER
;     -> DELIMITER

Identifier pattern:
letter(letter|digit)*
```

Q11. Write the Regular Expressions.

```
1. Starts with a and ends with b:
   a(a|b)*b

2. Exactly two a's over {a,b}:
   b*ab*ab*
   (The question appears to contain a typo if it says
    exactly two s's while the alphabet is {a,b}.)

3. Third character from the left is a:
   (a|b)(a|b)a(a|b)*

4. Binary number starts with 1 and ends with 0:
   1(0|1)*0

5. Contains three consecutive a's:
   (a|b)*aaa(a|b)*

6. Ends with abb:
   (a|b)*abb
```

Q12. Minimize the given DFA.

The actual DFA diagram is not included in the question-bank file, so the exact minimized DFA cannot be determined without inventing missing transitions. Use this procedure when the diagram is supplied:

```
Step 1: Initial partition
```

```
P0 = { Final states, Non-final states }

Step 2: Refine each partition.
For every pair of states in the same block, compare
their transitions for every input symbol.

Step 3: Split states that transition into different
partitions.

Step 4: Repeat until no partition changes:
P0 -> P1 -> P2 -> ... -> Pk

Step 5: Merge all states in each final partition.
The resulting DFA is the minimum equivalent DFA.
```

Q15. Convert the given NFA into an equivalent DFA.

The question bank states that a diagram will be given. Use subset construction; do not invent the missing NFA.

```
1. Start with the NFA initial state (and epsilon-closure if needed).
2. For each input symbol, find every reachable NFA state.
3. Treat each resulting set as one DFA state.
4. Repeat until no new sets are produced.
5. A DFA state is final if its set contains at least one NFA final state.

Key idea:
One DFA state represents a SET of NFA states.
```

Q16. Explain firstpos(), lastpos() and followpos().

firstpos(n): The set of positions that can appear as the first symbol of strings generated by node n.

lastpos(n): The set of positions that can appear as the last symbol of strings generated by node n.

followpos(i): The set of positions that can immediately follow position i in a string generated by the regular expression.

```
For a.b, with position(a)=1 and position(b)=2:

firstpos = {1}
lastpos  = {2}
followpos(1) = {2}

For concatenation X.Y:
for every i in lastpos(X):
followpos(i) = followpos(i) U firstpos(Y)
```

UNIT 3 — Syntax Analysis

Q17. Explain Augmented Grammar, Kernel Items and Viable Prefix.

Augmented Grammar: If S is the original start symbol, introduce a new start symbol S' and add S' -> S. This gives a unique start production and makes the accepting condition of LR parsing clear.

Kernel Items: The augmented initial item S' -> .S and LR items whose dot is not at the leftmost position.

Viable Prefix: A prefix of a right-sentential form that can appear on the stack of a shift-reduce parser; equivalently, it does not extend beyond the right end of a handle.

Q18. Demonstrate LALR Parsing for the given grammar.

The grammar is to be supplied in the exam. The correct construction is:

```
1. Augment the grammar: S' -> S
2. Construct canonical LR(1) item sets using closure and goto.
3. Construct transitions among item sets.
4. Find states with identical LR(0) cores.
5. Merge those LR(1) states and combine their lookaheads.
6. Construct ACTION and GOTO tables.
7. Parse the input using shift, reduce, accept and error actions.

An LR(1) item has the form:
[A -> alpha . beta, a]

LALR states are formed by merging LR(1) states
having the same LR(0) core.
```

Q19. Explain the method to construct an LL(1) Parser.

LL(1) means: input is scanned Left-to-Right, a Leftmost derivation is produced, and one symbol of lookahead is used.

```
Step 1: Remove left recursion.
Example:
E -> E + T | T
becomes
E  -> T E'
E' -> + T E' | epsilon

Step 2: Apply left factoring where required.

Step 3: Compute FIRST sets.

Step 4: Compute FOLLOW sets.

Step 5: Construct predictive parsing table M[A,a].

Step 6: Use the stack, input buffer and parsing table
to perform the parse.

Accept when:
Stack = $
Input = $
```

Q20. Construct an SDD that translates arithmetic expressions from infix to prefix notation.

```
L -> E
     print(E.val)

E -> E + T
     E.val = '+' E1.val T.val

E -> E - T
     E.val = '-' E1.val T.val
```

```
E -> T
     E.val = T.val

T -> T * F
     T.val = '*' T1.val F.val

T -> T / F
     T.val = '/' T1.val F.val

T -> F
     T.val = F.val

F -> F ^ P
     F.val = '^' F1.val P.val

F -> P
     F.val = P.val

P -> (E)
     P.val = E.val

P -> digit
     P.val = digit.lexval
```

Example: a + b * c -> + a * b c (or +a*bc). The operator is placed before its operands.

Q21. Apply Syntax-Directed Definition to construct a Desk Calculator.

```
L -> E n
     print(E.val)

E -> E1 + T
     E.val = E1.val + T.val

E -> T
     E.val = T.val

T -> T1 * F
     T.val = T1.val * F.val

T -> F
     T.val = F.val

F -> ( E )
     F.val = E.val

F -> digit
     F.val = digit.lexval
```

Example: 3 * 5 + 4 -> 15 + 4 -> 19, so E.val = 19.

Q22. Comparative analysis of Lexical, Syntax and Semantic Errors.

Error	Meaning	Example
Lexical	Invalid character/sequence prevents a valid token from being formed.	int @x;
Syntax	Tokens do not follow the grammar.	a = b +
Semantic	Program structure is valid but its meaning is invalid.	Incompatible type assignment.

Easy memory rule: Lexical = Is the token valid? Syntax = Is the structure valid? Semantic = Does the statement make sense?

Q23. Perform left factoring.

```
Given:
A -> xByA | xByAzA | a
B -> b
```

```
Common prefix = xByA

Left-factored grammar:
A  -> xByAA' | a
A' -> zA | epsilon
B  -> b
```

Left factoring removes common prefixes so that a predictive parser can choose the correct production using limited lookahead.

Q24. Explain Sentential Leftmost Derivation, Rightmost Derivation and Parse Tree.

```
Grammar:
S -> S + S | S - S | S * S | S / S | (S) | a

String:
a * a - a

Leftmost derivation:
S
=> S - S
=> S * S - S
=> a * S - S
=> a * a - S
=> a * a - a

Rightmost derivation:
S
=> S - S
=> S - a
=> S * S - a
=> S * a - a
=> a * a - a
```

Parse tree: A tree representing the syntactic structure of a derived string. Interior nodes are non-terminals and leaves are terminals. A syntax tree is a more compact abstract representation in which operators are internal nodes and operands are leaves.

Q25. Explain Shift-Reduce Parsing.

Shift-reduce parsing is a bottom-up parsing technique. It starts with input symbols and repeatedly finds handles and reduces them until the start symbol is obtained.

Shift: Move the next input symbol onto the stack. Reduce: Replace a handle on the stack by the corresponding left-hand-side non-terminal. Accept: The start symbol has been obtained and input is exhausted. Error: No valid parser action is available.

```
Typical example for id + id * id:

Shift id
Reduce F -> id
Reduce T -> F
Reduce E -> T
Shift +
Shift id
Reduce F -> id
Reduce T -> F
Shift *
Shift id
Reduce F -> id
Reduce T -> T * F
Reduce E -> E + T
Accept
```

A handle is a substring matching the RHS of a production whose reduction corresponds to one step in the reverse of a rightmost derivation. Handle pruning means finding and reducing such handles.

Q26. Draw Syntax Tree for the given Regular Expression.

The actual regular expression is not included in the question-bank file; it says that a syntax tree is to be drawn for a supplied regular expression. Therefore the exact tree cannot be determined here without inventing the missing expression.

General rule: operators such as |, ., and * become internal nodes, while operands/symbols become leaves. For a supplied expression, first parse its operator precedence and then build the corresponding tree.

High-Priority Exam Revision

- Phases of Compiler
- Compiler vs Interpreter
- DFA vs NFA
- Tokens, Lexemes and Patterns
- Regular Expressions
- DFA Minimization
- NFA to DFA
- Augmented Grammar, Kernel Items and Viable Prefix
- LL(1) Parser, FIRST/FOLLOW and parsing table
- Infix to Prefix Syntax-Directed Definition
- Desk Calculator SDD
- Left Factoring
- Leftmost/Rightmost Derivation and Parse Tree
- Shift-Reduce Parsing
- LALR construction

Important note: Q12, Q15, Q18 and Q26 require a specific DFA/NFA/grammar/regular expression that the question bank indicates will be supplied separately. This document therefore gives the method rather than fabricating a unique final diagram/table.